CoRec: An Efficient Internet Behavior-based Recommendation Framework with Edge-cloud Collaboration on Deep Convolution Neural Networks

YANGFAN LI and KENLI LI, College of Information Science and Engineering, Hunan University
WEI WEI, School of Computer Science and Engineering, Xi'an University of Technology
TIANYI ZHOU, Institute of High Performance Computing, A*STAR
CEN CHEN, College of Information Science and Engineering, Hunan University

Both accurate and fast mobile recommendation systems based on click behaviors analysis are crucial in e-business. Deep learning has achieved state-of-the-art accuracy and the traditional wisdom often hosts these computation-intensive models in powerful cloud centers. However, the cloud-only approaches put significant computational pressure on cloud servers and increase the latency in heavy-load scenarios. Moreover, existing work often adopts RNN structures to model behaviors that suffer from low processing speed for under-utilization of parallel devices such as GPUs. In this work, we propose an efficient internet behavior-based recommendation framework with edge-cloud collaboration on deep CNNs (CoRec) to improve both the accuracy and speed for mobile recommendation. A novel convolutional interest network (CIN) improves the accuracy by modeling the long- and short-term interests and accelerates the prediction through parallel-friendly convolutions. To further improve the serving throughput and latency, a novel device-cloud collaboration strategy reduces workloads by pre-computing and caching long-term interests in the cloud offline and real-time computation of short-term interests in devices. Extensive experiments on real-world datasets show that CoRec significantly outperforms the state-of-the-art methods in accuracy and has achieved at least an order of magnitude improvement in latency and throughput compared to cloud-only RNN-based approaches for long behaviors.

CCS Concepts: • **Information systems** → **Users and interactive retrieval**; **Data mining**; • **Applied computing** → **Electronic commerce**; • **Computing methodologies** → *Machine learning;*

The research was partially funded by the National Key R&D Program of China (Grant No. 2018YFB1003401), the National Outstanding Youth Science Program of National Natural Science Foundation of China (Grant No. 61625202), the International (Regional) Cooperation and Exchange Program of National Natural Science Foundation of China (Grant Nos. 61661146006, 61860206011), the Singapore-China NRF-NSFC Grant with No. NRF2016NRF-NSFC001-111, the Natural Science Foundation of Hunan Province under Grant 2020JJ5083, and the Cultivation of Shenzhen Excellent Technological and Innovative Talents (Ph.D. Basic Research Started) RCBS20200714114943014, by the National Natural Science Foundation of China under Grant 61902120, by the Cultivation of Shenzhen Excellent Technological and Innovative Talents (Ph.D. Basic Research Started) RCBS20200714114943014, and by the Basic research of Shenzhen Science and technology Plan under Grant JCYJ20210324123802006.

Authors' addresses: Y. Li and K. Li (corresponding author), School of Computer Science and Engineering, Central South University, No. 932 South Lushan Road, Changsha Hunan 410083 P.R. China; emails: yangfanli@hnu.edu.cn, lkl@hnu.edu.cn; C. Chen, Infocomm for Research Institute (I2R), Singapore, and Shenzhen Institute, Hunan University, China; email: chenc@i2r.a-star.edu.sg; W. Wei, School of Computer Science and Engineering, Xi'an University of Technology, Xi'an, China; email: weiwei@xaut.edu.cn; T. Zhou, A*STAR Centre for Frontier AI Research (CFAR), Singapore; email: zhouty@ihpc.a-star.edu.sg.

1550-4859/2022/12-ART24 $15.00
https://doi.org/10.1145/3526191

Additional Key Words and Phrases: Click through rate prediction, convolutional neural networks, internet of behaviors, edge-cloud collaboration

ACM Reference format:
Yangfan Li, Kenli Li, Wei Wei, Tianyi Zhou, and Cen Chen. 2022. CoRec: An Efficient Internet Behavior-based Recommendation Framework with Edge-cloud Collaboration on Deep Convolution Neural Networks. *ACM Trans. Sen. Netw.* 19, 2, Article 24 (December 2022), 28 pages.
https://doi.org/10.1145/3526191

1 INTRODUCTION

With the development of **Internet of Things (IoT)**, the close-to-user devices and sensors at the network edge including smartphones, smart wearable devices, smart cameras, and so on, provide valuable information about user interests, behaviors, and preferences and lead to the emergence of the **Internet of Behaviors (IoB)**. IoB facilitates a wide range of applications, such as smart health, smart transportation, internet services, and so on. Among these applications, internet behavior-based recommendation systems, which aim at displaying suitable items to users based on the analysis on behavior patterns, is an important life service [36]. Accurate and fast recommendation not only enhances user experience to recommend exactly what they need in real-time, but also increases the advertisement revenue of publishers such as Google and Amazon.

The key problem to provide accurate and fast recommendation is **click-through rate (CTR)** prediction, which is to estimate the ratio of a user to click or visit on a given item (locations or goods) [40]. CTR prediction models have evolved from traditional linear and non-linear models such as **Logistic Regression (LR)** [40, 43] and **Factorization Machines (FM)** [38] to deep learning-based methods [15, 44, 45, 51] for higher accuracy. The most effective idea of them to improve the accuracy is to capture the user interest from historical user behaviors to help CTR prediction and most of them adopt the structure of **recurrent neural network (RNN)** to model the behavior records. DIEN [51] suggests that user interest is dynamic and evolving and proposed an interest extraction layer based on GRU with attention mechanism for CTR prediction. DISN [46] proposes that user interest evolves among sessions and employed the Bi-LSTM to capture the session interest.

As calculating deep learning models is computation-intensive, the common approach often resorts to the powerful cloud center to host these computations [14, 31]. However, the cloud-only approach puts significant computational pressure on the cloud servers and the system throughput will become the bottleneck for providing service. In heavy-load scenarios such as festivals where a large number of user requests come in concurrently, the computation capacity of servers can be easily overwhelmed, and the requests can not be processed promptly, leading to high delay and low user experience. With the burst of performance of modern mobile SoCs, partitioning the workload across the cloud and mobile device is a promising option to alleviate server pressure and increase the system throughput. Researchers have proposed general frameworks [20, 21, 23] as well as task-specific models [37] to enable the collaborative device-cloud execution of deep learning algorithms. However, the throughput and latency improvement that they bring on CTR prediction tasks are quite limited, as the workload of the CTR prediction is workloads-proportional to the sequence length, which they do not manage to reduce and they process the entire sequence on the fly. This motivates us to answer a question: *How to execute the CTR prediction tasks collaboratively on the smart edge devices and cloud with high system throughput and low latency?*

Another problem that hinders real-time recommendation for existing CTR algorithms is most of them adopt the structure of RNN for interest modeling with low processing speed, especially when processing long records to leverage the long-term dependencies to improve the accuracy. It is well known [3, 47] that RNNs have a record-by-record sequential reliance to process the records,

therefore are hard to exploit the parallel computing units such as GPU in the cloud. The computation time therefore grows linear with the length of records. For example, it causes a delay of over 100 ms to execute DIEN on a cloud server equipped with Tesla P100 GPU to handle hundreds of behaviors for a single user.

Meanwhile, unlike RNNs, which have a sequential reliance in the computation pattern, convolutions can be done in parallel, since the same filters are used in each layer [16]. Currently, **convolution neural network (CNN)** has achieved comparable performance and has been proven to be time-efficient in many sequence modeling tasks, such as audio synthesis, word-level language modeling, and machine translation [3, 12, 16, 22]. The interest modeling problem in CTR prediction can also be formulated as a sequence modeling task, where we aim to extract user interest from user historical sequential behaviors. But the current sequence CNN models are limited to obtain a rich interest representation, which contains the long-term static interest such as brand and price, as well as their short-term dynamic interest to improve the prediction accuracy. Our second problem arises: *How to leverage the parallel-friendly convolution for CTR prediction and improves the accuracy?*

Aiming at these two problems, this work proposes a deep CNN-based recommendation framework with edge-cloud collaboration to provide both accurate and fast CTR prediction and high system throughput. We first propose **Convolutional Interest Network (CIN)**, a novel pure CNN-based algorithm for CTR prediction. The following strategies are proposed to improve the prediction accuracy based on the ordinary 1D CNN for sequence modeling: (1) A convolutional interest extraction component to capture the long- and short-term user interest. The insight behind is that the feature map of each convolutional layer can intrinsically represent a user's interest during the time period within the receptive field of this layer. Then, we propose a weighted temporal layer aggregation scheme that aggregates these latent interest representations to describe the overall long- and short-term user interest. (2) An interest supervision loss component to learn better representations of user interest in the training process. We observe the fact that the user current interest will lead to the successive behavior and we are inspired by the negative sampling scheme in Reference [33]. Based on the observation and inspiration, we suppose that interest representations generated by the convolutional interest extraction component are good representations if they can help to distinguish the successive positive clicked target from random draws of uninteresting data (un-click item noises). Following this idea, we design the interest supervision loss where negative behavior samples and the user profile are used to supervise the training process of the whole network.

To further alleviate the pressure of servers, reduce the serving latency, and increase the system throughput, we propose a novel device-cloud collaboration strategy. We first analyze the computation pattern of CIN and observe that only a small proportion of workload for short-term interest is required to be done online and most of workload for long-term interest can be pre-computed and cached offline. Based on this observation, we propose the CIN partitioning and execution strategy with online workload reduction that reduces the online serving workload by pre-computing and caching the long-term interest representations offline in the cloud and offload the computation of short-term interest to the device. The proposed incremental feature computing strategy further reuses the intermediate result for continuous recommendation.

The contributions of this article are summarized as follows:

- We propose **convolution interest network (CIN)**, a novel CNN-based algorithm that can make both accurate and fast CTR prediction by analyzing user behaviors.
- We propose a novel edge-cloud collaboration strategy for CIN to reduce the recommendation serving latency by reducing the online workload and utilizing the edge device to analyze the short-term user behaviors.

- Extensive experiments on the real-world Amazon and Alibaba datasets and real hardware platform have demonstrated that our CoRec significantly outperforms the state-of-the-art methods in terms of accuracy and has achieved at least an order of magnitude improvement in latency (compared to RNN-based models) and throughput (compared to cloud-only approach) in dealing with long record sequences.

The remaining of the work is organized as follows: In the next section, the related work is discussed. Section 3 describes the overview of CoRec. Section 4 presents the details of algorithm design of CoRec. Section 5 presents our edge-cloud collaborative strategy. Section 6 presents the experimental results, while the work is concluded in Section 7.

2 RELATED WORK

In this section, we introduce studies on CTR prediction, sequence models, layer aggregation scheme, and negative sampling loss.

2.1 Traditional CTR Prediction Models

Traditional linear or non-linear methods such as **logistic regression (LR)** [40, 43] and **Factorization Machines (FM)** [38] have been widely used for CTR prediction. In Reference [4], authors handcraft many features from raw data and use LR to predict the click-through rate. Reference [6] also uses LR to deal with the CTR problem and scales it to billions of samples and millions of parameters on a distributed learning system. In Reference [34], a Hierarchical Importance-aware **Factorization Machine (FM)** [39] is introduced, which provides a generic latent factor framework that incorporates importance weights and hierarchical learning. In Reference [13], boosted decision trees have been used to build a prediction model. In Reference [18], a model that combines decision trees with logistic regression has been proposed and outperforms either of the above two models. However, these methods often need a lot of feature engineering effort, which is often performed by domain expert. Besides, they also leave the historical sequential dependencies unemployed.

2.2 Deep Learning Models for CTR Prediction

Recently, inspired by the great success of deep learning in **computer vision (CV)** [19], **natural language processing (NLP)** [2] and various domains [9, 48, 49], deep methods for CTR prediction have also achieved significant improvement. Deep learning-based methods for CTR prediction mainly fall into the following two categories:

The first category focuses to learn the implicit interactions between features. Wide&Deep [10] and xDeepFM [25] combined low- and high-order features for a better performance. PNN [35] proposed a product layer to capture interactive patterns between inter-field categories. DIFMs [28] adaptively reweight the original feature representation and flexibly learns more informative features. **Convolutional neural network (CNN)** has also been used for capturing the feature interactions. FGCNN [27] performed convolutions to capture the interactions between adjacent features such as user, item type, time, query, and so on. However, the CNN-based methods are sensitive to how the features are aligned. In summary, there is room for improvement of performance for these models, as they do not utilize the temporal dependencies in users' historical data.

The second category aims to model the temporal dependencies from historical user interactions, which is more relevant to our method. DREAM [45] suggested that RNN helps a lot to learn the dynamic representation of each user. HierTCN [44] combined GRU and TCN to model the user behaviors. DIEN [51] proposed an interest extract layer based on GRU with attention mechanism to extract the evolving user interest. DSIN [15] designed a self-attention network to get accurate interest representations of each session and utilized the Bi-LSTM to capture sequential dependencies.

SURGE [5] employs the graph neural network to capture the long-term preferences from behaviors. These methods improved the accuracy of CTR prediction a lot compared to sequence-independent methods significantly. However, most of these approaches utilized RNN to model the temporal dependencies, which may be a big obstacle to deploy these methods on web-scale applications, as RNNs are hard to parallelize [50]. Our proposed CIN is an interest model like DSIN and DIEN. CIN is fully CNN-based and has shown to be much more faster than RNN-based methods.

2.3 Executing Deep Neural Networks in Edge-cloud Environment

There are three main categories of approaches to execute DNN models in edge-cloud environment.

The most common way is to execute the DNN tasks entirely on the cloud server for inference [14, 31]. However, the cloud-only approach puts significant computational pressure on the cloud servers, and the system throughput will become the bottleneck for providing service.

The second category executes all DNN computations on the edge device side. However, this design requires extensive expert experience and skills to design DNNs with low accuracy loss (e.g., MobileNet, ShuffleNet, and so on) [41], or various compression technologies (e.g., low-rank factorization, knowledge distillation, quantization, and pruning) to reduce unimportant weights and neurons of trained models to acquire lightweight DNNs [21]. Moreover, the capacity of edge device is small and cannot perform large models and big data input in a reasonable latency.

The third category implements collaborative inference between the edge device and the cloud server by dynamically partitioning DNNs, which leverages available resources of the edge device, reduces the load of the cloud center, and somewhat accelerates DNNs inference. Researchers have proposed general frameworks [20, 21, 23] as well as task-specific models [37] to enable the collaborative device-cloud execution of deep learning algorithms. Neurosurgeon [23] is a classic device-cloud collaborative DNN inference scheme that automatically chooses the partition points pursuing the optimal latency and energy consumption. LcDNN [20] and DeepAdapter [21] provide lightweight collaborative frameworks for executing distributed DNN inference between the mobile web and edge server, which also rely on the edge server. Reference [42] further extends the collaborators, which jointly train mapped sections of a DNN onto a distributed computing hierarchy over the cloud, the edge server, and end devices. However, most of them assume that the entire input to be processed online, which put huge strain on the computing frameworks in this big data era [7, 8]. We belong to this category and design more deliberate partition and caching strategy to reduce the online workload and latency. We adopt the design without edge server, because the edge server requires real-time interaction with the cloud based on the user request in the CTR prediction stage and improves little on the performance.

3 FRAMEWORK OVERVIEW

We first propose a novel algorithm, termed as **convolutional interest network (CIN)** for CTR prediction. CIN is purely CNN-based and runs fast by exploiting the parallel computing units such as GPUs to extract the user interest from historical user behavior records. Meanwhile, CIN improves the accuracy by modeling the long- and short-term interest in the behaviors records.

Based on the proposed CIN, we design a recommendation framework with edge-cloud collaboration shown in Figure 1 to further alleviate the pressure of servers and reduce the latency of the traditional cloud-only in the heavy load scenarios with a large number of concurrent user requests. The overall workflow is illustrated as follows:

- **Training CIN in the cloud.** The cloud server is responsible for the training process with the proposed interest supervision loss. The trained interest extraction model and embeddings of candidate items are dispatched to the edge devices.

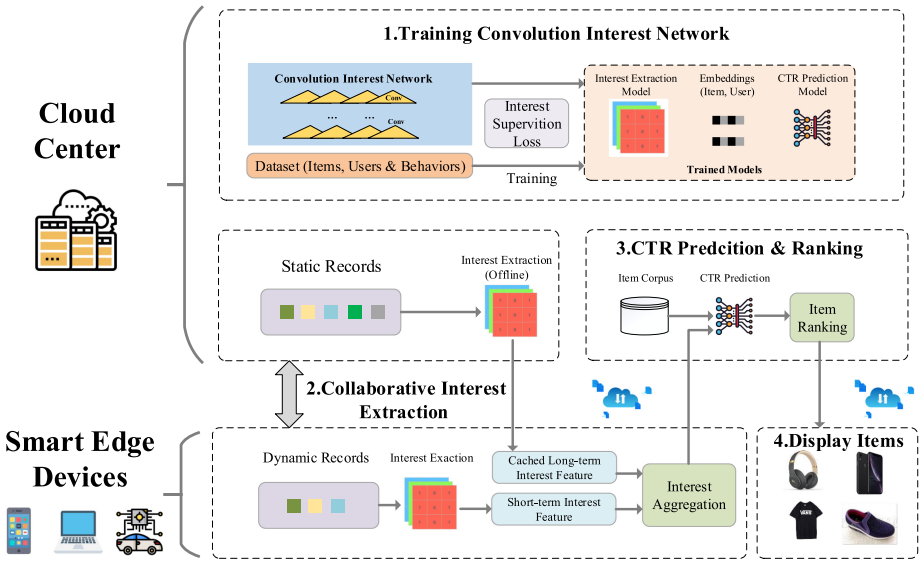

Fig. 1. Overview of the deep convolutional neural networks-based recommendation framework with edge-cloud collaboration.

- **Edge-cloud collaborative interest extraction.** The most workload of CIN comes from extracting the user interest from user behaviors. The collaborative strategy contains two phases for interest extraction: (1) The offline stage executed in the cloud aims at capturing the long-term interest in advance and cache the intermediate long-term interest representations. Most of the computation workload of CIN can be done in the offline stage. (2) The online stage extracts the short-term interests based on the real-time user clicks and the cached representations. We use the devices to host the online stage, as only very little computations are involved and the device-generated request can be handled with no data transmission overhead.
- **CTR prediction, item ranking, and displaying.** The trained CTR prediction model calculates the CTR of each candidate item selected from the item corpus. As the candidate selection strategy is dynamically controlled by the publisher, the CTR prediction phase should be executed on the cloud. Then, the candidate items are ranked by CTR. Finally, the top-rank items are displayed on the device.

4 ALGORITHM DESIGN OF COREC

In this section, we propose a novel algorithm, i.e., **convolutional interest network (CIN)** that can model long- and short-term user interest in historical user behaviors to accurate **click through rate (CTR)** prediction. CIN is purely CNN-based and does not have any sequential reliance in the computation patterns, which enables exploiting highly parallel computation and collaborative execution in the cloud-device environment.

4.1 Problem Definition and Algorithm Overview

For e-commerce platforms such as Amazon and Alibaba, the display of advertisements (or goods) often follows a two-stage information retrieval dichotomy [52] where CTR prediction plays an important role. Figure 2 briefly illustrates the whole procedure. The first stage is the candidate

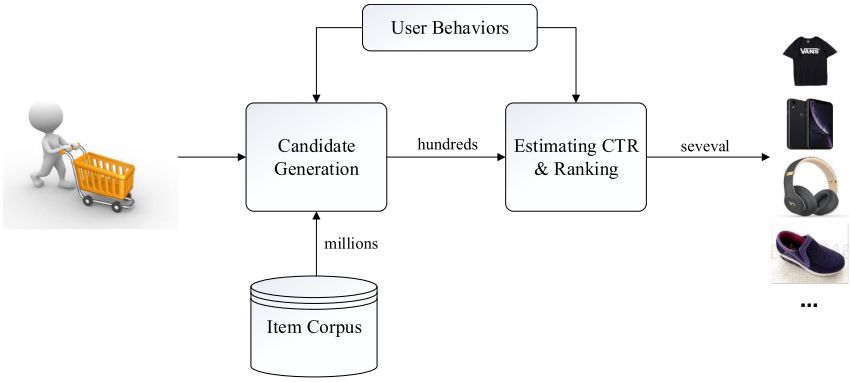

Fig. 2. Illustration of the display of items in e-commerce platforms. CTR prediction model plays an important role.

generation stage, where a relatively small set of candidates relevant to the user are generated from the large corpus via methods such as collaborative filtering. The second stage is the ranking stage. CTR for each candidate in the small set from the previous stage is predicted by the system during this stage. The items are then ranked according to CTR and some other factors according to the policies of publishers such as bids by advertisers. Finally, several items with higher ranks are displayed for users. In this two-stage framework, the CTR has a strong impact on the ranking of items, thus affects both the user experience and the profit of publishers (e.g., Amazon).

In this article, we focus on the problem of CTR prediction, which is crucial and most computational-intensive in recommendation systems. Similar with References [40, 51], our goal is to estimate the probability of a user to click on a displayed target item (advertisement or good). Specifically, we have three feature groups: the user profile denoted as U, the user behaviors sequence denoted as B, and the target item profile denoted as I. Items can be advertisements or goods in different datasets. Each feature group contains several features. User profile contains user_id, user_gender, user_age, and so on. Target item profile contains item_id, item_category, predicted_time, and so on. User behaviors sequence includes a sequence of click item_id, item_category, click time, and so on. Then, the CTR prediction problem is defined as follows:

$$CTR = p(action = click|U, I, B). \tag{1}$$

Our proposed CIN is a deep neural network to estimate the probability. The network structure is shown in Figure 3. The inputs of CIN are user profile U, user behaviors sequence B, and target item profile I. The output is the predicted CTR. CIN consists of four components: (1) a feature embedding component, (2) a convolutional interest extraction component, (3) an interest supervision loss component, (4) and an output component. The details are presented as follows:

- In the feature embedding component, we embed all the raw features into low-dimensional dense vector representations. The inputs are all the raw features, i.e., U, I, and B, discussed above. The outputs are all the embedded features.
- In the convolutional interest extraction component, we extract the long- and short-term user interest from historical user behaviors. The input is the behaviors sequence embedding, and the output is a corresponding sequence of dense vector representations of user interest.
- In the interest supervision loss component, we design the objective function to train the whole network. With the carefully designed objective function, CIN can learn the user interest more effectively, thus make better predictions.

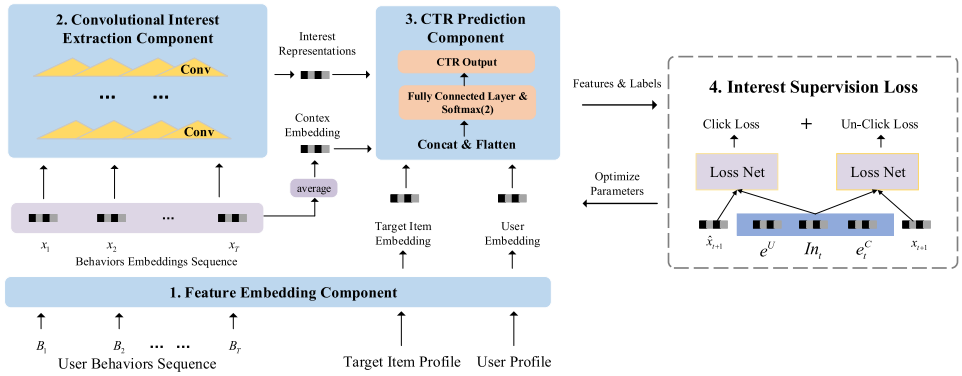

Fig. 3. Network structure of convolutional interest network (CIN).

- In the output component, we use the user embedding, the target item embedding, the context embedding, and the interest representation at the last timestamp to calculate the final CTR.

4.2 Feature Embedding Component

The feature embedding component transforms all the raw features into low-dimensional dense vector representations that are more representative for neural networks to process. We also extract the session information from the user historical behaviors as the model feature. The embeddings are saved after training to denote the features of item/user for online serving.

4.2.1 Session Information Extraction. The user behaviors can be naturally divided into different sessions. Their behaviors are highly homogeneous in each session and heterogeneous across sessions. Some work such as HierTCN [44] and DSIN [15] designed specialized network to incorporate session information, but the specialized network is too complicated and increases the computation. We consider the session information as a feature of a behavior directly, which is equally helpful for the CNN to learn the session information. More specifically, for every behavior, we mark this behavior "inter-session" if the time interval between this behavior and the former latest behavior is smaller than a given threshold (this threshold is manually selected, usually one hour for most datasets), elsewise "cross-session." Following this rule, the session information can be extracted from the user behaviors directly.

4.2.2 Feature Encoding. We encode all the raw features as high-dimensional sparse one-hot or multi-hot vectors, as the raw features cannot be processed directly. Most of the features in CTR prediction tasks are categorical, such as gender, type, and so on. The categorical features are usually encoded as one-hot vectors or multi-hot vectors. For example, the gender feature is encoded as $[0, 1]$ for male and $[1, 0]$ for females. After encoding, the user profile is denoted as $\boldsymbol{U} = \{\boldsymbol{U}_1, \boldsymbol{U}_2, \ldots, \boldsymbol{U}_{N^U}\}$, where $\boldsymbol{U}_i$ denotes an encoded sparse feature for users and N^U represents the number of features in the feature group user profile. Similarly, Target Item Profile is denoted as $\boldsymbol{I} = \{\boldsymbol{I}_1, \boldsymbol{I}_2, \ldots, \boldsymbol{I}_{N^I}\}$. User behaviors sequence is a list of user behaviors and is denoted as $\boldsymbol{B} = \{\boldsymbol{B}_1, \boldsymbol{B}_2, \ldots, \boldsymbol{B}_T\}$, where T is the length of the behaviors sequence. $\boldsymbol{B}_t$ denotes the user behavior at timestamp t, which contains a set of features, such as id, category, user rating, and so on. Session sequence is sequence of session labels denoted as $\boldsymbol{S} = \{\boldsymbol{S}_1, \boldsymbol{S}_2, \ldots, \boldsymbol{S}_T\}$, where $\boldsymbol{S}_t$ denotes the session label regarding the user behavior $\boldsymbol{B}_t$. The feature set used in CIN is reported in Table 1.

4.2.3 Embedded Feature Representation. After encoding, the raw input features transform to high-dimensional sparse vectors. But these high-dimensional sparse vectors are less representative

Table 1. Feature Set Used in CIN

Feature Group	Feature Name	Feature Dimension	Feature Type
User Profile	user_gender	2	one-hot
	user_age	~10	one-hot
	user_id	$\sim 10^7$	one-hot
	...	...	...
User Behaviors	clicked_item_id	$\sim 10^7$	one-hot
	clicked_item_category	$\sim 10^5$	one-hot
	user_rating	~10	multi-hot
	...	...	...
Item Profile	item_id	$\sim 10^7$	one-hot
	item_category	$\sim 10^5$	one-hot
	...	...	...

for neural networks to process. To tackle this, we introduce an embedding layer to transform them into low-dimensional dense vectors. In the embedding layer, each feature in the same feature group is transformed into a dense vector that has the same dimension L. Then, concatenating all the vectors of the same group forms the representation of this group. Formally, the embedding operation $\psi : \mathbb{R}^D \rightarrow \mathbb{R}^L$ is a linear transformation that transforms the sparse vector $\boldsymbol{s}_i \in \mathbb{R}^D$ into a dense vector $\boldsymbol{e}_i \in \mathbb{R}^L$. The embedding transformation is formulated as follows:

$$\boldsymbol{e}_i = \psi(\boldsymbol{s}_i) = \boldsymbol{s}_i^{\mathrm{T}} \boldsymbol{W}, \tag{2}$$

where $\boldsymbol{W} \in \mathbb{R}^{D \times L}$ is a learnable parameter matrix and i denotes the ith feature in a feature group. Then the feature group $\boldsymbol{e} \in \mathbb{R}^{N \cdot L}$ is

$$\boldsymbol{e} = \{\boldsymbol{e}_1 \circ \boldsymbol{e}_2 \ldots \circ \boldsymbol{e}_N\}, \tag{3}$$

where N denotes the number of features in a feature group (user Profile or target item or so on) and $\circ$ denotes the concatenation operation.

With the embedding layer, the four feature groups, i.e., user profile, target item profile, and user behaviors sequence can be represented as $\boldsymbol{e}^U$, $\boldsymbol{e}^I$, $\boldsymbol{e}^B = \{\boldsymbol{e}_1^B, \boldsymbol{e}_2^B, \ldots, \boldsymbol{e}_T^B\}$, respectively.

Moreover, considering the fact that all the behaviors actually reflect the overall preference of a user, we average all the behaviors embedding to form the context information of a user, denoted as

$$\boldsymbol{e}_t^C = average\left(\boldsymbol{e}_1^B, \boldsymbol{e}_2^B, \ldots, \boldsymbol{e}_t^B\right), \tag{4}$$

where $\boldsymbol{e}_t^C$ is the average of the behavior embedding before time t, representing the context information at time t.

4.3 Convolutional Interest Extraction Component

Our proposed convolutional interest extraction component can extract the long- and short-term user interest from historical user behaviors via convolution operation. In this section, first, we define the problem of interest extraction. Second, we propose a basic CNN structure for interest extraction. Our convolutional interest extraction component is built on the basic CNN structure. Third, we analyze how to model long- and short-term user interest using the hierarchical feature maps of CNN. Finally, we formally present the formulation of our convolutional interest extraction component.

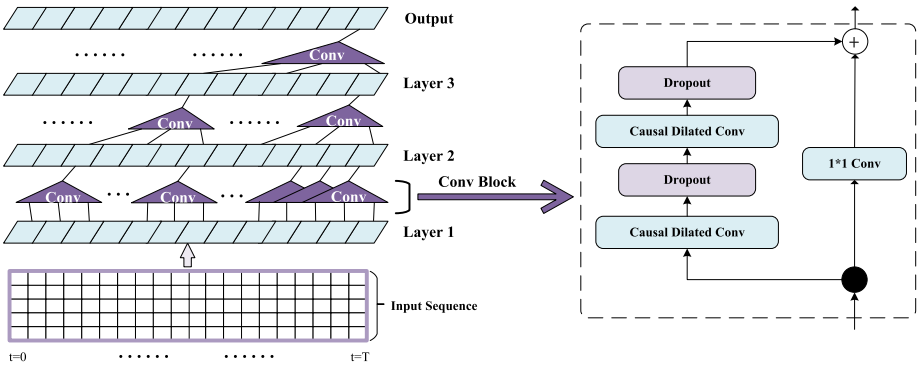

Fig. 4. Basic CNN structure for interest extraction. (a) An example of the convolutional operation on the input sequence. (b) Block structure and residual connection.

4.3.1 Problem of Interest Extraction. Before presenting our network structure, we talk about the interest extraction problem. In our CTR prediction task, user behaviors are the carrier of user interest, which is crucial to make accurate CTR prediction. We need to extract the user interest from the behaviors. Considering the fact that every successive behavior of a user is motivated by his current interest, the interest model should output an interest representations sequence corresponding to the behaviors sequence. Each element in the interest representations sequence denotes the interest representation that can help to predict the successive behavior. It is desirable to formulate the interest extraction problem as a sequence modeling problem as follows:

$$\boldsymbol{In}_t = g(\boldsymbol{x}_{1:t}), \tag{5}$$

$$\{\boldsymbol{In}_1, \boldsymbol{In}_2, \ldots, \boldsymbol{In}_T\} = \{g(\boldsymbol{x}_1), g(\boldsymbol{x}_{1:2}), \ldots, g(\boldsymbol{x}_{1:T})\}, \tag{6}$$

where $\boldsymbol{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_T\}$ is the input sequence describing user behaviors, $\boldsymbol{In}_t$ is the extracted interest representations at time t, and $g(\cdot)$ is the network we need to train following the idea that $\boldsymbol{In}_t$ can help predicting $\boldsymbol{x_{t+1}}$ (the objective function will be discussed later). Moreover, no information leakage is allowed for $\boldsymbol{In}_t$, i.e., $\boldsymbol{In}_t$ should only depend on $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_t\}$, and no information leakage from $\{\boldsymbol{x}_{t+1}, \ldots, \boldsymbol{x}_T\}$ is allowed according to Equation (5).

4.3.2 Basic CNN Structure for Interest Extraction. We have formulated the interest extraction problem as a sequence modeling problem. We first propose a basic CNN structure shown in Figure 4 to solve the problem. The basic CNN structure evolves from the 1D CNN, which is the most common way for CNN-based sequence modeling. The 1D CNN is formulated as follows:

$$F(\boldsymbol{x}_t) = (\boldsymbol{X} * \boldsymbol{f})(t) = \sum_{j=0}^{k-1} \boldsymbol{f}_j \cdot \boldsymbol{x}_{t-j}, \tag{7}$$

$$\hat{\boldsymbol{Y}} = \{\hat{\boldsymbol{Y}}_1, \hat{\boldsymbol{Y}}_2, \ldots, \hat{\boldsymbol{Y}}_T\} = \{F(\boldsymbol{x}_1), F(\boldsymbol{x}_2), \ldots, F(\boldsymbol{x}_T)\}, \tag{8}$$

where $\boldsymbol{x}_t \in \mathbb{R}^d$ denotes the tth element in the input sequence $\boldsymbol{X}$, d denotes the dimension of input elements, $\boldsymbol{f} \in \mathbb{R}^{k \times d}$ denotes a convolution kernel with size k, and d is the number of kernels. Stacking several layers then forms a deep network.

However, the ordinary 1D CNN is not applicable for interest modeling for two reasons: (1) the length of the output will shrink so 1D CNN cannot output an interest representations sequence of the same length as the input. (2) The receptive field is linear to the depth of the network. The network will become very deep to tackle with long sequence, which will bring huge computation.

To address the problems, we introduce causal convolution, dilated convolution, and residual connections in the ordinary 1D CNN. Causal convolution is used to ensure no future information leakage, while dilated convolution and residual connections are often used for larger receptive field in many CNN-based methods for sequence modeling tasks [3, 12].

Causal Convolution. In the interest modeling problem mentioned above, the length of the inputs and outputs should be the same while no future information leakage is allowed. To achieve the two goals, causal convolution uses zero padding to guarantee the length of the input and output sequence are the same. For the second goal, the causal convolution operation at time t does not involve the input elements after time t. The causal convolution is formulated as follows:

$$F_c(\boldsymbol{x}_t) = (\boldsymbol{X} * \boldsymbol{f})(t) = \sum_{j=0}^{k-1} \boldsymbol{f}_j \cdot \boldsymbol{x}_{t-j}, \boldsymbol{x}_{\leq 0} \coloneqq 0, \tag{9}$$

$$\hat{\boldsymbol{Y}}^c = \{\hat{\boldsymbol{Y}}_1^c, \hat{\boldsymbol{Y}}_2^c, \ldots, \hat{\boldsymbol{Y}}_T^c\} = \{F_c(\boldsymbol{x}_1), F_c(\boldsymbol{x}_2), \ldots, F_c(\boldsymbol{x}_T)\}. \tag{10}$$

Dilated Convolution. To enlarge the receptive field for processing longer historical behaviors, dilated convolutions are employed. As shown in Figure 4(a), the upper layers have larger dilation factors and the bottom layer has the dilation factor of 1, which means a regular convolution. With the expanding dilation factor, the top layer can access a much longer history. Formally, a causal dilated convolution layer is formulated as follows:

$$F_{dc}(\boldsymbol{x}_t) = (\boldsymbol{X} *_{l_r} \boldsymbol{f})(t) = \sum_{j=0}^{k-1} \boldsymbol{f}_j \cdot \boldsymbol{x}_{t-l_r \cdot j}, \boldsymbol{x}_{\leq 0} \coloneqq 0, \tag{11}$$

$$\hat{\boldsymbol{Y}}^{dc} = \{\hat{\boldsymbol{Y}}_1^{dc}, \hat{\boldsymbol{Y}}_2^{dc}, \ldots, \hat{\boldsymbol{Y}}_T^{dc}\} = \{F_{dc}(\boldsymbol{x}_1), F_{dc}(\boldsymbol{x}_2), \ldots, F_{dc}(\boldsymbol{x}_T)\}, \tag{12}$$

where l_r denotes the dilation factor of the rth layer.

Residual Connection. If we want to further extend the receptive field to leverage more long-term temporal information and model long-term interest, then more convolutional layers are required and the network will become even deeper then the performance will go down. The residual learning and linear shortcut connections can help to solve the exploding and vanishing gradient problems in the long-term back-propagation [17], especially in the networks with deeper structures. With the benefits of residual neural networks [17], we introduce residual connections into our basic CNN structure as shown in Figure 4(b). Two causal dilated convolution layers described in Equations (12) and (11) form a block. The residual connection is formulated as follows:

$$\boldsymbol{O}_{i+1} = ReLU(\boldsymbol{O}_i + \xi(\boldsymbol{O}_i)), \tag{13}$$

where ξ denotes the function of the ith block, $\boldsymbol{O}_i$ denotes the input of the ith block, and $\boldsymbol{O}_{i+1}$ denotes the output of the ith block and the input of the $(i+1)$-th block. The **rectified linear unit (ReLU)** is utilized as the activation function. An additional 1×1 convolution is utilized to ensure that element-wise addition receives tensors of the same shape.

In our basic CNN structure, we stack several blocks to form the hierarchical architecture.

4.3.3 Analysis on Feature Map of CNN and User Interest. Based on the proposed basic CNN structure, we analyze how to model the long- and short-term interest to further improve the performance. The basic CNN structure does not have the inductive bias over the users' long-term and static preference, as well as the short-term dynamic intention discussed in the introduction. Unlike in RNN where the more previous cell has lower gradient, all the elements in the behaviors sequences are treated the same by a same set of convolution kernels. The result is that there is no actual difference between a behavior long ago and a recent behavior for the basic CNN structure,

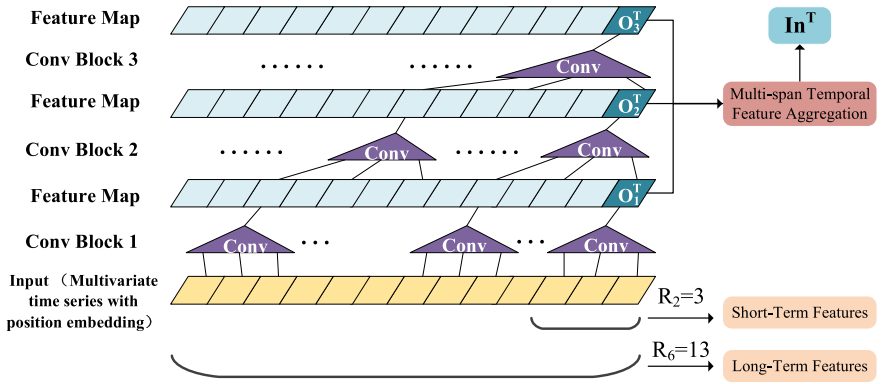

Fig. 5. Feature map of each layer can intrinsically represent a user's interest during the time interval within the receptive field of this layer. Leveraging all these feature maps can describe long- and short-term user interest.

but indeed the two behaviors have very different impacts on the user's future action. To tackle this problem, we investigate how to model the long- and short-term user interest via convolutions.

One way to impose the inductive bias over the long- and short-term user interest is using different neural networks to model different kinds of interest. For example, LSTNet [24] used an RNN and a hop-RNN in parallel to model the short- and long-term temporal patterns, respectively. A similar scheme of paralleling multiple basic CNN structures can also be applied to model the different interests. However, this idea may not meet the requirement for fast processing in CTR prediction tasks, because paralleling multiple networks means that the parameters of the network will also multiply and consequently increase the processing speed considerably [26].

Rather than designing a network with multiple CNNs in parallel, we aim to use a single CNN modeling long- and short-term interest mainly for higher processing speed. Motivated by FPN [26], which suggested that all the feature maps of different convolutional layers are semantically strong, we further generate an insight that the feature map of each layer can intrinsically represent a user's interest during the time interval within the receptive field of this layer. Figure 5 illustrates this idea. The detailed explanation is given in the following:

Feature Maps and User Interest. For a basic CNN structure with several convolutional layers (the bottem layer is the first layer), the receptive field R_j of the jth layer is calculated as follows:

$$R_j = 1 + \sum_{i=0}^{j} D_i \cdot (K_i - 1), \tag{14}$$

where D_i is the dilation factor of the ith layer, and K_i is the kernel size of the ith layer. This equation indicates that the upper layers have much larger receptive fields than the lower layers, which suggests that the feature maps of lower layers can represent the short-term intention, while the feature maps of upper layers can represent long-term preference intrinsically.

For example, Figure 5 illustrates a basic CNN structure with six layers (three residual blocks), where kernel sizes $K_i = 2$ for all the six layers, and dilation factor $\boldsymbol{D} = \{1, 1, 2, 2, 3, 3\}$ for the six layers, respectively. Each triangle represents a residual convolutional block with two convolutional layers. One block can access three elements. According to Equation (14), the receptive field $\{R_2, R_4, R_6\}$ turns out to be $\{3, 7, 13\}$. Then, part of the feature maps of the three layers, i.e., $\boldsymbol{O}_1^T$, $\boldsymbol{O}_2^T$,

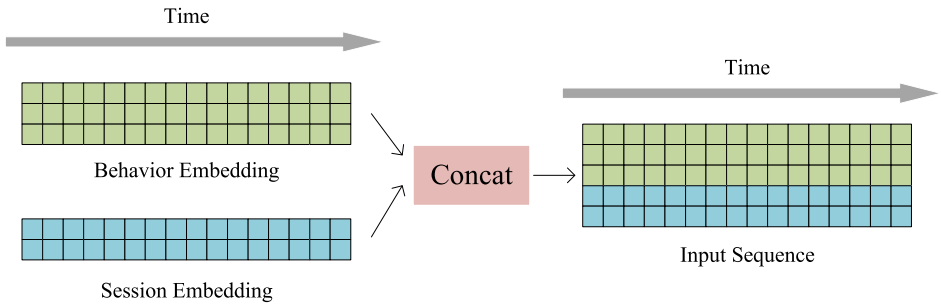

Fig. 6. Illustration of how our network is aware of the session information. The sessions embeddings sequence is concatenated with the behaviors embeddings sequence to form the input sequence.

and $\boldsymbol{O}_3^T$, can represent user interest of the most recent 3, 7, 13 behaviors, respectively. Fusing the three latent vectors can represent users' overall interest at time T.

4.3.4 Formulation of Convolutional Interest Extraction Component. Based on the discussions above, our convolutional interest extraction component can be formally presented in detail.

Session Awareness. First, in Figure 6, the sessions embeddings sequence is concatenated with the behaviors embeddings sequence to let our network be aware of the session information. In Section 4.2, we have the behaviors embeddings sequence $\boldsymbol{e}^B$. As $\boldsymbol{e}^B$ does not contain the session information, we extract the session embedding sequence information $\boldsymbol{e}^S$. For every behavior $\boldsymbol{e}_t^B$, there is a corresponding session information $\boldsymbol{e}_t^S$. It is considerable to concatenate the two sequences to let our network be aware of the session information for every behavior. Formally, the input sequence is formulated as follows:

$$\boldsymbol{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_T\} = \left\{\boldsymbol{e}_1^B \circ \boldsymbol{e}_1^S, \boldsymbol{e}_2^B \circ \boldsymbol{e}_2^S, \ldots, \boldsymbol{e}_T^B \circ \boldsymbol{e}_T^S\right\}, \tag{15}$$

Basic CNN Structure for Interest Extraction. Second, we use the basic CNN structure proposed in Section 4.3.2 to process the above input behavior embedding sequence $\boldsymbol{e}^B$ and extract the interest. The output of the basic CNN structure is denoted as follows:

$$\boldsymbol{O} = \Phi(\boldsymbol{X}), \tag{16}$$

where Φ denotes the basic CNN structure. Shown in Figure 5, $\boldsymbol{O} = \{\boldsymbol{O}_1, \boldsymbol{O}_2, \ldots, \boldsymbol{O}_{N^B}\}$ is a set of feature maps of the blocks where $\boldsymbol{O}_i$ is the feature map of the ith block. N^B denotes the number of blocks. $\boldsymbol{O}_i^t$ denotes part of the feature map of the ith block at timestamp t.

Weighted Temporal Layer Aggregation for Long- and Short-term Interest. We propose the weighted temporal layer aggregation scheme to fuse the feature maps generated by the basic CNN structure to model the long- and short-term interest. Formally, the interest representation of long- and short-term interest at timestamp t is as follows:

$$\boldsymbol{In}_t = W_1 \cdot \boldsymbol{O}_1^t \circ W_2 \cdot \boldsymbol{O}_2^t \circ \cdots \circ W_{N^B} \cdot \boldsymbol{O}_{N^B}^t, \tag{17}$$

where $W_i \in \mathbb{R}$ is a trainable weight and $\circ$ is the concatenation operator. We introduce the weight parameter, because in real-world scenarios, the degree of influence of different layers on user interest may be different. Each element in $\boldsymbol{In} = \{\boldsymbol{In}_1, \boldsymbol{In}_2, \ldots, \boldsymbol{In}_T\}$ represents user interest at every timestamp.

4.4 Output Component

The final output estimated CTR is formulated as follows:

$$CTR = \xi\left(\boldsymbol{e}^U, \boldsymbol{e}_T^S, \boldsymbol{e}^I, \boldsymbol{e}_T^C, \boldsymbol{In}_T\right). \tag{18}$$

We concatenate and flatten the user embedding $\boldsymbol{e}^U$, the target item embedding $\boldsymbol{e}^I$, the session embedding $\boldsymbol{e}_T^S$ at the last timestamp T, the context embedding $\boldsymbol{e}_T^C$ at time T, and the interest representation $\boldsymbol{In}_T$ at time T. $\xi(\cdot)$ denotes the concatenating and flattening operation, and a fully connected layer with softmax output.

4.5 Interest Supervision Loss Component

In this paragraph, we propose the interest supervision loss component, which supervises the training process to learn better interest representations.

The simplest object function is to minimize the negative log-likelihood function, which uses the label of the target item to supervise the training:

$$L_{target} = -\frac{1}{N_s} \sum_{(x,y)\in D_{train}}^{N_s} (y \log(CTR) + (1-y)\log(1-CTR)), \tag{19}$$

where $x = \{I, B, U\}$ is the training samples, D_{train} is the training set of size N_s, $y \in \{0, 1\}$ denotes whether the user clicks the target item.

However, only minimizing L_{target} means that only the interest representation at the final timestamp $\boldsymbol{In}^T$ is employed while the former interest representations $\{\boldsymbol{In}_t | t < T\}$ do not contribute to the training process. Motivated by word2vec [33], which suggested that a latent representation is good if it can distinguish the positive samples from noises with a simple model, we propose interest supervision loss for CTR prediction. We observe the fact that the user current interest will lead to the successive behavior. Based on this observation, we suppose that interest representations generated by the convolutional interest extraction component are good representations if they can help to distinguish the successive positive clicked target from random draws of uninteresting data (un-click item noises) with a simple model such as a shallow network. DIEN [51] proposed a similar scheme employing negative samples. Our scheme is distinct from them, as they do not consider the user profile for generating the interest representation, which is a flaw for learning the interest for a specific user. The experiments have also proven our superiority.

The details of interest supervision loss are shown in the following: In the training process, together with the user behaviors sequence B, we generate a negative behavior sequence $\hat{B}$ with the same length as B where each element $\hat{B}_i$ satisfies: $\hat{B}_i \neq B_i$. The item $\hat{B}_t$ represents an item that a user will not click at timestamp t. Then, $\hat{B}$ is embedded into $\hat{\boldsymbol{e}}^B$ via the feature embedding component proposed in Section 4.2. Concatenating $\hat{\boldsymbol{e}}^B$ with the session embedding sequence forms a negative input sequence $\hat{\boldsymbol{X}}$. Then, the negative sampling loss is formulated as follows:

$$\begin{aligned} L_{neg} = -\frac{1}{N}\Bigg(\sum_{i=1}^{N_s} \sum_{t=1}^{T-1} \Big(& \log\left(\delta(\boldsymbol{e}_t^C, \boldsymbol{In}_t, \boldsymbol{e}^U, \boldsymbol{x}_{t+1})\right) \\ & + \log\left(1 - \delta\left(\boldsymbol{e}_t^C, \boldsymbol{In}_t, \boldsymbol{e}^U, \hat{\boldsymbol{x}}_{t+1}\right)\right)\Big)\Bigg), \end{aligned} \tag{20}$$

where $\delta(\cdot)$ denotes a small fully connected network with sigmoid activation. L_{neg} contains two terms, which means that we aim to optimize two targets simultaneously. Given $\boldsymbol{e}^U$, which is the latent representation of a specific user and $\boldsymbol{e}_T^C$ representing the context of the user, the first term suggests that we want $\boldsymbol{In}_t$, which is the latent representation of user interest at timestamp t, can

help to distinguish the positive clicked item at the next timestamp $t + 1$, while the second term suggests we want to distinguish the not-clicked item that the user is not interested in at time $t + 1$.

The final objective function is then formulated as follows:

$$L = L_{target} + \alpha \cdot L_{neg}, \tag{21}$$

where α is the hyper-parameter that balances the interest representation and CTR prediction.

Our CIN is end-to-end trainable, which means that all the parameters are optimized together to minimize the loss in function Equation (21).

4.6 Advantages of CIN Comparing with RNN-based Methods for Interest Modeling

Comparing with the recurrent structure such as RNN, GRU, and LSTM, which are used for interest modeling in DIEN [51] and DSIN [46], our basic CNN structure is a fully convolutional structure that has the following main advantages:

- **High processing speed.** RNN is hard to parallelize, since it has to wait for the predecessors completed before the next procedure, while convolutions can be done in parallel, since the same filters are used in each layer. Both the training phase and the testing phase can benefit from higher processing speed in GPU because of parallelism.
- **Flexible memory of historical behaviors input.** The memory of RNN is often unpredictable and short because of the gradient vanishing problem. So, RNN cannot deal with the long-term dependencies well. Some variants such as LSTM and GRU only alleviate this problem. In CIN, the length of the memory is determined by the receptive field, which can be adjusted according to different scenarios. Our proposed weighted temporal layer aggregation scheme enables the network to memorize both the long- and short-term dependencies to achieve better accuracy.

5 EDGE-CLOUD COLLABORATIVE EXECUTION STRATEGY OF CIN

Although the proposed CIN yields a much lower computational latency to handle a single request by exploiting the cross-behavior parallelism than the RNN-based methods, the latency still increases dramatically to deal with a large amount of concurrent user requests in the cloud-only approach. We propose a novel device-cloud collaboration strategy to alleviate the pressure of servers and reduce the recommendation serving latency. The proposed workload partitioning and offloading strategy greatly reduces the online serving workload and enables leveraging the computation capacity of smart edge devices. The proposed incremental feature computing strategy further reuses the intermediate result for continuous recommendation.

5.1 CIN Partitioning and Execution Strategy with Online Workload Reduction

The most workload of CIN comes from extracting the user interest from user behaviors. A natural approach to provide online recommendation serving with CIN is to extract the user interest from the entire behaviors upon each request in the cloud. In this manner, the server will be stressful to handle a large number of concurrent user requests and the delay will increase. To alleviate the pressure of cloud center, the traditional partitioning approaches [30] often use row/column- and grid-based strategies for distributed CNN inference, However, they often assume the entire inputs are dynamically generated and do not consider the static characteristics of historical records. Therefore, the computation workload for them is still too large to provide real-time inference.

To tackle this issue, we propose an elaborate strategy to partition and execute the workload of extracting the user interest with two stages shown in Figure 7: (1) The offline stage executed in the cloud aims at capturing the long-term interest in advance and cache the intermediate long-term interest representations. Most of computation workload of CIN can be done in the offline stage.

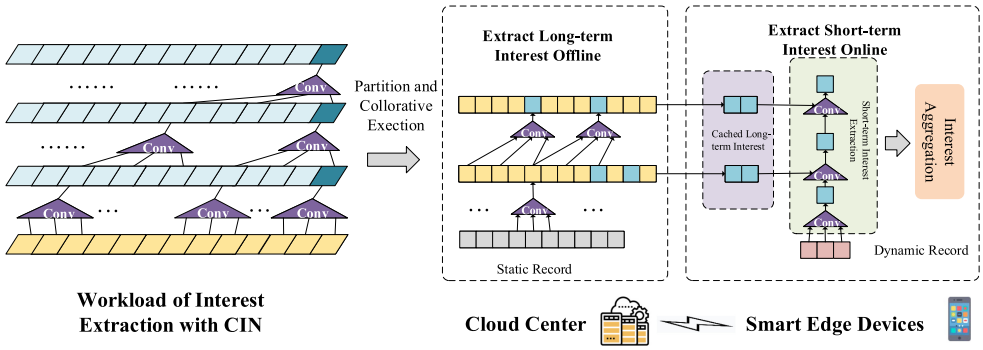

Fig. 7. The offline phase holds most of the computation of CIN in the cloud. In the online phase, the user requests are dynamically computed combined with the cached result in the offline phase with an adaptive device-cloud workload partition strategy.

(2) The online stage extracts the short-term interests based on the real-time user clicks and the cached representations. We use the devices to host the online stage, as only very few computations are involved and the device-generated request can be handled with no data transmission overhead. In this way, most of the workload is pre-computed in the offline stage and the online workload is greatly reduced and executed on devices to alleviate server pressure for lower latency. The details are illustrated as follows:

5.1.1 Static and Dynamic Records. We first partition the user behavior records as static records and dynamic records. The dynamic records are dynamically generated on user devices based on the real-time click behaviors and are processed in real time in the online stage on the device, while the static records have been generated for a time and are often saved in the cloud for long-term interest extraction in the offline stage. The length of the dynamic record is chosen as the size of the convolution kernels to enable a complete convolution operation on the device. Therefore, most of the records are static records, as the kernel size in CIN is usually small. For example, the common kernel size (dynamic record length) is from 3 to 5 while the entire record length ranges from tens to hundreds.

5.1.2 Extracting Long-term Interest Offline (Cloud). The offline phase aims at extracting and caching the long-term interest from the static records. CIN uses causal convolution to extract the interest representations from the records. As no future records are involved in causal convolutions, the long-term interest representations (feature maps) only depend on the static records and will not change regardless of future behaviors, which enables pre-computing and caching them as intermediate results (feature maps). The cached intermediate results of long-term interest representations can be reused in the online stage to reduce the real-time workload for serving.

Based on this analysis, in the offline phase before the online user request, we compute and cache the feature maps generated from the static records if they have not been calculated before. Note that not all the feature maps are required to be pre-computed, as they will never be used when new requests arrive (the yellow feature maps in Figure 7). Formally, given the input embedding of a static records $\boldsymbol{e}^{B_s}$ that contains T_s elements, we use CIN to compute the feature map H^i of the ith convolution layers. In the original CIN, H^i also contains T_s elements, but only part of the features need to be computed in the offline phase. The interval of H^i can be calculated as follows:

$$I_i = 1 + \sum_{j=0}^{i} D_j \cdot (K_j - 1), \tag{22}$$

where D_j is the dilation factor of the jth layer, and K_j is the kernel size of the jth layer. The sequence of layers follows a top-to-down manner. Then, the feature maps that need to be calculated in the ith layer is $\{H_j^i | T_s - I_i < j \leq T_s\}$. Given an input behavior sequence with length T and CIN with N layers (the same kernel size and numbers) and receptive field R, the remained workload is proportion to N, as we only need to calculate the convolution for the last timestamp. The total workload for the ith layer is $\min(T, R - D_i \cdot (K_i - 1))$. The portion of remained workload p can be calculated as follows theoretically:

$$p = \frac{N}{\sum_{i=0}^{N} \min(T, R - D_i \cdot (K_i - 1))}. \tag{23}$$

For example, in Figure 7, the historical records marked as grey are the static records, and the records in red are dynamic records. To generate to final interest representation, the required feature maps are marked, which also falls into two categories: the first kind of feature map is generated from the static records and can be pre-computed and cached for reuse while second kind of feature map relies on the dynamic records.

5.1.3 Extracting Short-term Interest Online (Device). The online stage extracts the short-term interests based on the real-time user clicks and the cached representations. We use the devices to host the online stage, as only very few computations are involved and the device-generated request can be handled with no data transmission overhead. The workload in the online phase to calculate the CTR given the dynamic user request at time T (most recent behavior) and the cached feature maps generated in the offline phase are given as follows: (1) calculating the feature map of each layer at time T. In this stage, the input of the convolution operation of the ith layer can be explicitly figured out. It contains the feature maps at T of the $i-1$-th layer, which need to be dynamically computed, and $K_i - 1$ feature maps at the previous time steps (static records), which have been generated in the offline stage. (2) Aggregate the long- and short-term interest representations through the proposed weighted temporal layer aggregation scheme. (3) Calculate the CTR through fully connected layer.

5.1.4 Incremental Feature Computing. Another scenario in real-world recommendation is continuous recommendation. In this occasion, the dynamic records become longer with continuous user behaviors and increase the workload of online stages. We propose an incremental strategy to further reuse the feature maps generated in the online phase for continuous recommendation. In the online phase, when calculating the CTR, we also generate the feature maps of the dynamic records. For the following user requests, the dynamic records are treated as static records and these feature maps will also be required. Therefore, we manage the feature maps in an incremental way, which incrementally caches the feature maps once the user requests have been processed. In this way, the workload of online stage on the devices can be fixed in the continuous recommendation scenarios.

5.2 Device-cloud Collaborative Recommendation Algorithm

Based on the above discussion, the device-cloud collaborative recommendation algorithm is formally shown in Algorithm 1. Lines 1–4 describe the offline phase on the cloud. For each selected user, CIN is performed on the static records to extract the long-term interest. These interest representations are cached and dispatched to the user devices when applicable, for example, when the APPs are launched. Lines 5–13 present the online phase on the device for a specific user. If the device has already cached the long-term interest, then the required features of dilated convolutions are first selected. The short-term interest representations are extracted based on the selected features and the dynamic records. The final interest representation is generated by aggregating

ALGORITHM 1: Device-cloud Collaborative Recommendation Algorithm

Input: Static Records B_s; Dynamic Records B_d; Candidate Item Set I; User Set U.
Output: User Interest Representation In.

▷ Offline Phase (Cloud)

1: **for** each $u \in U$ **do**
2: $long\_term\_interest \leftarrow Cloud\_CIN(B_s)$;
3: $Dispatch\_For\_User(long\_term\_interest, u)$;
4: **end for**

▷ Online Phase (Device)

5: **if** $long\_term\_interest$ is valid **then**
6: $selected\_feature \leftarrow Select\_Feature(long\_term\_interest)$;
7: $short\_term\_interest \leftarrow Device\_CIN(B_d, selected\_feature)$;
8: $In \leftarrow Interest\_Aggregation(short\_term\_interest, selected\_feature)$;
9: **else**
10: $In \leftarrow Cloud\_CIN(B_d, B_s)$;
11: **end if**
12: $long\_term\_interest \leftarrow Incremental\_Feature\_Computing(long\_term\_interest, short\_term\_interest)$;
13: **return** In;

both the long- and short-term features. In line 12, the newly generated short-term interest representations are cached in an incremental way to enable the efficient continuous recommendation. Note that the final CTR computing and recommendation stage based on extracted interest are not presented in this algorithm, as it is not computational-intensive and should always be executed on the cloud based on the cloud item corpus. In addition, CoRec is a scalable design to support the workload reduction and the collaborative execution scheme to work independently. It is feasible for CoRec to process the short-term interest in the cloud to support the edge devices with very low processing ability and when the server is not busy. It can bring improvement in latency by offloading the short-term interest calculating back to the cloud, as the cloud servers are more powerful, but at a cost of significant reduction in system throughput.

6 EXPERIMENTS

We describe our experiment in this section to serve two purposes. First, we evaluate the performance of our proposed CIN. We have conducted extensive experiments with seven baselines (including DIEN, the state-of-the-art model) on three real-world datasets for CTR prediction. The experimental results have demonstrated that CIN significantly outperforms them in terms of accuracy. Second, we evaluate the system performance of CIN and the proposed device-cloud collaborative execution strategy for CTR prediction serving in real hardware platform, and the results indicate that we have achieved at least an order of magnitude improvement in latency (compared to RNN) and throughput (compared to cloud-only approach) in dealing with long record sequences.

6.1 Evaluate Algorithm Performance

We evaluate the AUC of the proposed CIN with seven baselines (including DIEN, the state-of-the-art model) on three real-world datasets for CTR prediction. The experimental results have demonstrated that CIN significantly outperforms them in terms of accuracy. Besides, we have conducted ablation studies to verify the effectiveness of our proposed layer aggregation scheme and interest supervision loss. We have visualized and analyzed the learned latent vector representations to further demonstrate the effectiveness of CIN.

Table 2. Dataset Details

Dataset	Users	Goods	Categories	Samples
Amazon(Books).	603,668	367,982	1,601	8,898,041
Amazon(Electronics).	192,403	63,001	801	7,824,482
Alibaba	281,042	128,921	1,023	9,826,571

6.1.1 Dataset Description. We use three real-world datasets, including two Amazon datasets and one Alibaba dataset. Table 2 illustrates the corpus statistics. Training dataset is generated with $k = 1, 2, \ldots, n-1$-th behaviors for each user. In the test/validation set, we predict the last one given the first $n - 1$ reviewed goods. The last behaviors of all the users are divided into test/validation sets by half. All the datasets are publicly available.

Amazon dataset.[1] Amazon dataset contains product reviews and metadata from Amazon and is a benchmark dataset [17, 51, 52] for CTR prediction. In our experiments, we use two subsets, Books and Electronics, following DIEN [51]. In the two datasets, we regard goods as items and reviews as user behaviors. User profile U includes user_id. Item profile I includes item_id and category_id. User behaviors sequence B is a sequence of items reviewed by the user. Each review contains the item_id and category_id and an int rating score from 1 to 5 for the reviewed item. Supposing that we have T historical reviews for a user, our task is to predict the probability for the user to write reviews for a given item in his $(T + 1)$-th behavior.

Alibaba dataset.[2] Alibaba dataset is a public advertising dataset published by Alimama, an online advertising platform in China. Alibaba dataset has also been widely used for evaluating CTR prediction models. It contains the unclick/click records of users in eight days, from $2017 - 05 - 06$ to $2017 - 05 - 13$. In this dataset, we regard advertisements as items and the clicks on ads as user behaviors. U includes user_id, user_gender, user_age, user_city, and so on. I includes item_id, category_id, item_brand, and so on. User behaviors sequence is a sequence of ads clicked by this user. Each behavior contains all the clicked ad features the same as I. Supposing that we have T historical clicks for a user, our task is to predict the probability for a user to click on a given ad in his $(T + 1)$-th behavior.

6.1.2 Methods for Comparison. We have compared our CIN with the following mainstream methods for CTR prediction:

- **LR** [32]. Logistic Regression is a simple while widely used model for CTR prediction. LR serves as a weak baseline in our experiments.
- **TCN** [3]. TCN is a general CNN architecture for sequence modeling. TCN serves as a CNN baseline for CTR prediction.
- **GRU** [11]. GRU is a simple yet powerful variant of RNNs with gating mechanism for sequence modeling. GRU serves as an RNN baseline for CTR prediction.
- **Wide&Deep** [10]. Wide&Deep model is a widely used model in real industrial applications. It has two components. The first one is a wide model to handle the manually designed features. The second one is a deep model that automatically extracts nonlinear relations among features.
- **PNN** [35]. PNN introduced a product layer after embedding layer to capture high-order feature interactions.
- **DIN** [52]. DIN used an attention network to capture user interest from user behaviors.

[1] http://jmcauley.ucsd.edu/data/amazon/.
[2] https://tianchi.aliyun.com/dataset/dataDetail?dataId=56.

- **DIEN** [51]. DIEN proposed two RNN-based layers, i.e., an interest extractor layer and an interest evolving layer to extract user interest from user behaviors. DIEN is the state-of-the-art method.
- **CIN**. CIN is our proposed method.

6.1.3 Metrics. **AUC (Area Under ROC Curve)** is a widely used metric in the CTR prediction problem. AUC reflects the ranking ability of the model. AUC is defined as follows:

$$AUC = \frac{1}{m^+ m^-} \sum_{x^+ \in D^+} \sum_{x^- \in D^-} (I(f(x^+) > f(x^-))), \tag{24}$$

where D^+ is the collection of all positive examples, D^- is the collection of all negative examples, $f(\cdot)$ is the result of the model's prediction of the sample x, and $I(\cdot)$ is the indicator function.

6.1.4 Implementation Details. Tensorflow [1] was used to build our CIN and all the other comparing methods, including LR, TCN, GRU, Wide&Deep, PNN, DIN, and DIEN. All the hyper-parameters were selected via grid search. We trained and evaluated all the methods on a server with four Tesla P100 GPUs and eight E5-2620V4 CPUs.

For CIN, the implementation details are given as follows:

- **Feature Embedding Component.** As discussed in Section 4.1, we have three groups of features for all the datasets, i.e., user profile, target item profile, and user behaviors sequence. Each group has several features, such as user_id or item_id. In this component, each feature is transformed to a 18-dimensional vector. Then, the vectors in the same group are concatenated to form the representation of this group.
- **Convolutional Interest Extraction Component.** We stack 3 blocks, i.e., 6 convolutional layers, to process the user behaviors sequence. The numbers of kernels for all the layers are 100. The kernel size k is 5 for all the kernels. The dilation factors for the kernels of the 6 layers are $\boldsymbol{D} = \{1, 1, 2, 2, 4, 4\}$ where D_i denotes the dilation factor of the ith layer. We aggregate the outputs of the 3 blocks to represent the long- and short-term user interest. According to Equation (14), the receptive field of the first block turns out to be 9, which means the feature map of the first block represents user interest in his most recent 9 behaviors. Similarly, the feature map of the top block represents the most recent 57 behaviors.
- **Interest Supervision Loss Component.** In this component, the $\delta(\cdot)$ function in Equation (20) is a small fully connected network with three layers, which have 100, 50, and 2 neurons, respectively. The negative instances are randomly sampled from the item set.

For TCN, the same embedding component as CIN is also used to process the raw features. We stack six TCN layers to process the user behaviors sequence. The number of convolutional kernels, the size of kernels, and the dilation factors in TCN are all the same as in CIN. The same output component as CIN is used to generate the predicted CTR.

For GRU, the same embedding component with CIN is also used to process the raw features. We stack two GRU layers to process the user behaviors sequence. The dimension of the hidden state in GRU is 100. The same output component as CIN is used to generate the predicted CTR.

For Wide&Deep, PNN, DIN, and DIEN, we employ the public implementation[3] given by Reference [51].

We use the Adam optimizer to train CIN and all the other methods. The learning rate is 0.001 and the batch size is 128.

[3] https://github.com/mouna99/dien.

Table 3. Results (AUC) of Different Methods on Alibaba Dataset and Amazon Datasets

Model	Amazon Books	Amazon Electronics	Alibaba
LR	0.7216	0.7013	0.7398
TCN	0.7801	0.7413	0.7662
GRU	0.7691	0.7363	0.7574
Wide&Deep	0.7664	0.7400	0.7820
PNN	0.7758	0.7416	0.8061
DIN	0.7787	0.7526	0.8043
DIEN	0.8557	0.7583	0.7985
CIN (our model)	**0.8912**	**0.7674**	**0.8175**

Table 4. Performance Analysis of CIN

Model	Amazon Book	Amazon Electronic	Alibaba
base model	0.8030	0.7136	0.7615
base model+SE	0.8152	0.7278	0.7783
base model+TA	0.8454	0.7320	0.7885
base model+ISL	0.8609	0.7423	0.7987
base model+AL	0.8413	0.7311	0.7884
base model+SE+TA+ISL (CIN)	**0.8912**	**0.7674**	**0.8175**

SE denotes the proposed session embedding in the feature embedding component. TA denotes the temporal aggregation scheme in the convolutional component. ISL denotes that we use our proposed interest supervision loss scheme for training. AL denotes that we use the auxiliary loss proposed in DIEN for training. The proposed schemes can work together or independently. Best results are displayed in **boldface**.

6.1.5 Experimental Results. We report the experimental results of CIN and all the comparing methods on the three datasets. Table 3 reports the results on Alibaba dataset and Amazon datasets. We have repeated the experiments five times and report the average results. From this table, we find that all the deep models outperform the basic LR model significantly. PNN outperforms Wide&Deep, as PNN uses a specially designed scheme to capture feature interactions. DIN and DIEN are even better, as they learn the user interest from user behaviors. Among all the methods, our proposed CIN performs the best. CIN uses weighted temporal layer aggregation scheme to model the long- and short-term interest and the interest supervision loss to learn better representations of user interest, which improves the performance significantly.

6.1.6 Algorithm Performance Analysis. To study the performance and effectiveness of all the proposed schemes, we have conducted a set of ablation studies on Amazon and Alibaba datasets. The based model contains the feature embedding component, the residual causal convolution component, the output component, and the basic loss L_{target}. All these components constitute a basic end-to-end trainable framework to solve our problem. We then evaluate the effectiveness of the proposed schemes, including the session embedding in the feature embedding component, the temporal aggregation scheme in the convolutional component, and the negative sampling loss L_{neg}.

The experimental results are summarized in Table 4. We observe that all the proposed schemes improve the performance of base model and they can work together for even better improvements, which demonstrates the effectiveness of the proposed schemes. Among them, the interest supervision loss contributes most and demonstrates the importance of a good interest representation for CTR prediction. The temporal aggregation scheme also improves the performance

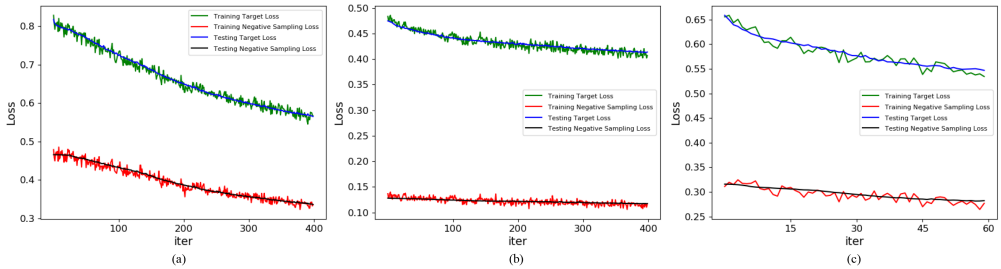

Fig. 8. Learning curves of CIN on public datasets. (a) Results on Amazon Books dataset. (b) Results on Amazon Electronics dataset. (c) Results on Alibaba dataset.

significantly, as it compensates for CNN's inability to model temporal information such as long- and short-term interest. The session embedding contributes least, but has still demonstrated its effectiveness.

We further compare the performance of the loss compared with the most similar auxiliary loss proposed in DIEN. Base model + ISL outperforms base model + AL, because we consider the user profile to supervise the training process to learn the interest for a specific user. Figure 8 illustrates the learning curves for CIN/ISL, where the target loss and the negative sampling loss both descend in a similar trend, which has also demonstrated the effectiveness of our proposed interest supervision loss.

6.1.7 Visualization. Finally, we have conducted a case study on the Amazon Book dataset to further verify the effectiveness of our proposed CIN.

Visualization of Latent Interest Representations. In Figure 9, we have visualized and analyzed the learned latent interest representations generated by DIEN and CIN of over 120K users from the test set on the Amazon Book dataset. The cluster property can measure the quality of the interest representations. As the user interest is complex and unlabeled, it is hard to analyze them very specifically. In another aspect, if the interest representations form clusters, then it is indicated that the users within a cluster may have similar interest. Interest representations with good cluster property can also be easily classified by algorithms. Therefore, representations with clearer boundaries can better preserve the user interest.

The results have proven CIN is more capable than DIEN on capturing the complex user interest. Figure 9(a) shows the interest representations in CIN, which are extracted by our proposed session-aware convolutional interest extraction component. Figure 9(b) shows the interest representations in DIEN, which are extracted by their RNN-based interest evolving layer. We have used t-SNE [29] to reduce the dimension of the latent interest representations from 36 to 2 for visualization. For CIN in Figure 9(a), we can see that the user interest naturally forms many clusters clearly, which proves that the clustering property of the learned interest is good. CIN can capture the complex interest of different users. For DIEN in Figure 9(b), the clustering boundaries are not clear, which means that the learned latent vectors are less representative than CIN.

Visualization of Item Embeddings. Learning good item embeddings is another key factor for accurate prediction. We have randomly selected 11 categories of books and visualized the embedding vectors with t-SNE in Figure 10. The categories include Literature & Fiction, Action & Adventure, Historical, Romance, and so on. Each point represents the embedded item_id. The points with the same color belong to the same category. We can see that the items belonging to the same category generate a clear cluster, which proves that our CIN can learn very good embedding vectors.

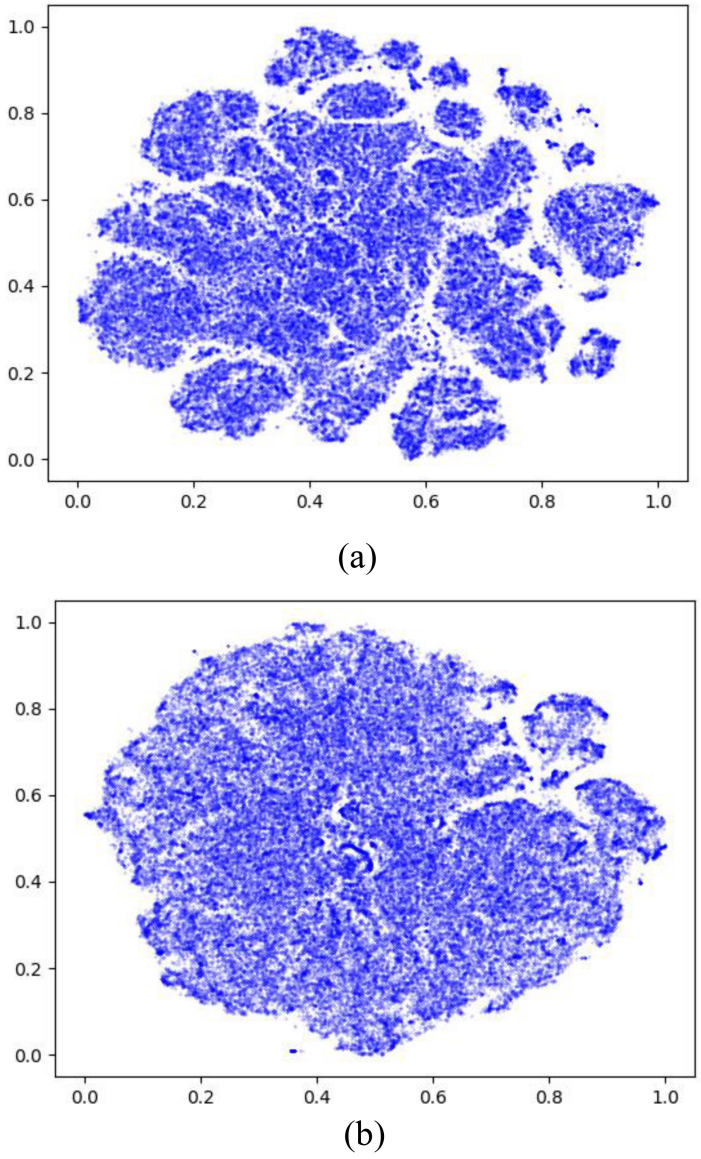

Fig. 9. Visualization of latent interest representations of over 120K users from the test set on Amazon Book dataset. Each point denotes the extracted latent interest representation of a user. Users within a cluster may have similar interest. Representations with clearer boundaries can better preserve the user interest. (a) Results of CIN. The user interest naturally forms many clusters clearly. (b) Results of DIEN. The clustering boundaries are not clear.

6.2 Evaluating System Performance

We evaluate the system performance of CIN, as well as the proposed device-cloud collaborative execution strategy in real hardware platform for CTR prediction serving, and the results indicate that we have achieved at least an order of magnitude improvement in latency (compared to RNN) and throughput (compared to cloud-only approach) in dealing with long record sequences.

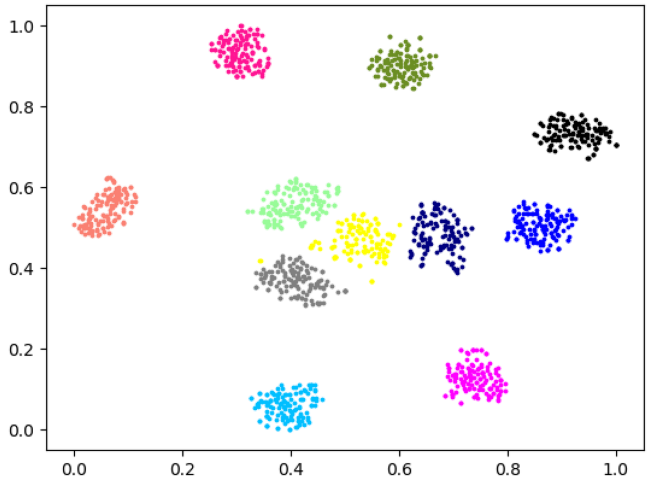

Fig. 10. Visualization of Item Embeddings. Each point represents the embedded item_id. The points with the same color belong to the same category.

6.2.1 System Settings. We use a real hardware platform to evaluate the system performance. We use a server with four Tesla P100 GPUs, eight E5-2620V4 CPUs, and 128 GB RAM to denote the cloud server. We use Jetson TX2, which has an NVIDIA GPU of the Pascal architecture with 256 CUDA cores, a quad-core ARM A57 CPU, and eight GB RAM to represent a smart edge device. The pre-trained models are executed on the cloud GPU and edge GPUs.

6.2.2 Methods for Comparison. We have two main objectives in this experiment. First, we want to figure out the system performance improvement of our CIN with the RNN-based methods in a cloud-only setting. Second, we want to know the improvement brought by the device-cloud collaborative execution strategy compared with the cloud-only setting for CIN. The following notations denote different experimental settings:

- **DIEN/Cloud** denotes executing the inference of DIEN, the state-of-the-art RNN-based methods for CTR prediction, in the cloud-only setting.
- **CIN/Cloud_LS** denotes executing our proposed CIN in the cloud setting, including processing both the long- and short-term interest.
- **CIN/Cloud_S** denotes executing our proposed CIN in the cloud setting with online workload reduction scheme, including only the short-term interest processing.
- **CIN/Collaborative** denotes executing the interest extraction stage of our proposed CIN with the device-cloud collaborative execution strategy. Note that the CTR prediction stage is still executed on the cloud, as the candidates are generally selected by the cloud center, which hosts the entire item corpus. Also, the system performance only includes the online stage, while the offline stage is assumed to be processed in advance.

We evaluate the system performance with different records length, denoting different workload settings. In the CTR prediction stage, the number of candidate items is 50, which is a common setting in the real-world recommendation system. The workload of generating these candidates is not considered in our work, as it is often a publisher-specific strategy.

6.2.3 Metric of System Performance. As the accuracy has been evaluated in the previous section, the device-cloud collaborative execution strategy of CIN will not introduce variance to the

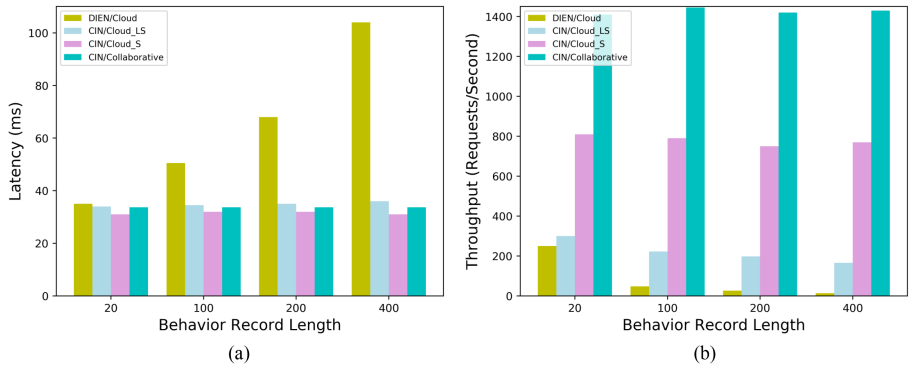

Fig. 11. System performance result for CTR prediction serving with different record length. We have achieved at least an order of magnitude improvement in latency (compared to DIEN, a state-of-the-art RNN-based method) and throughput (compared to cloud-only approach) in dealing with long record sequences. (a) Latency. (b) Throughput.

result in terms of accuracy. We measure the following two metrics that are highly relevant to user experience:

- **Latency** contains computation delay and communication delay. Computation delay is the execution time of the inference stage of evaluated models, including the interest extracting stage and the CTR prediction stage. We adopt a fixed 30 ms, a typical delay in the interest, as the communication delay.
- **Throughput** is defined as how many CTR prediction requests the system can process in a given amount of time. The throughput is measured in a single process per GPU way, where each GPU holds one copy of model and handles one request after another.

6.2.4 Result Analysis. The experimental results are shown in Figure 11, and we analyze the results as follows:

Latency Improvement. The latency results are shown in Figure 11(a). Comparing the performance of DIEN/Cloud and CIN/Cloud (including CIN/Cloud_LS and CIN/Cloud_S), we find that when the sequences are short, their latency is comparable. But with the longer sequences, the latency grows linearly with the sequence length while our CIN only has a slight increase in latency. This is mainly because in the interest extraction stage, the RNN-based method has a record-by-record sequential reliance to process the records that cannot be parallelized by the cloud GPU, while our CIN is CNN-based and leverage the cross-record parallelism with a higher hardware utilization. Comparing the performance of CIN/Cloud and CIN/Collaborative, we find that the latency of CIN/Collaborative is almost fixed and lower than CIN/Cloud because of our proposed reduced workload strategy and collaborative scheme to greatly reduce the required workload. But as the computation power of the mobile GPUs is much smaller than the cloud GPUs, the latency improvement is not significant to execute the reduced workload on mobile GPUs compared with executing the entire workload on the cloud. Indeed, we think the computational latency for both of the settings satisfy the requirement for real-time recommendation (less than 4 ms).

Throughput Improvement. The throughput results are shown in Figure 11(b). Comparing the performance of DIEN/Cloud and CIN/Cloud (including CIN/Cloud_LS and CIN/Cloud_S), we find in the cloud-only settings, the throughput of CIN is much larger than DIEN, because the GPU utilization of our convolution structure is much higher than the RNN-based structure. Comparing

the performance of CIN/Cloud and CIN/Collaborative, we find that the throughput improvement brought by the collaborative strategy is significant (at least an order of magnitude for long records). The reason comes from two folds. First, through our proposed collaborative scheme and workload reduction strategy, the workload that is required to be processed on the fly is greatly reduced and invariant to the record length. Second, the workload of interest extraction is offloaded to the device, leaving only the workload of CTR prediction in the cloud, which further increases the throughput by leveraging the computation power of multiple mobile edge devices.

Discussion. CoRec is a scalable design that supports processing the short-term interest in the cloud for edge devices with very low processing ability or when the server is not busy. It can bring improvement in latency by offloading the short-term interest calculating to the cloud as the cloud servers are more powerful, but at a cost of significant reduction in system throughput. Comparing the latency and throughput for Cloud_LS and CIN/Cloud_S, the computational delay has been reduced to a low level (<4 ms) relative to the communication delay (30 ms in the internet). Further reducing the computational latency contributes little to the user experience in terms of latency by offloading the computation to cloud servers. Meanwhile, the computation workload still remains a key factor for system throughput that is critical for user experience in heavy-load scenarios. Offloading the computation to the servers can reduce throughput by half. Furthermore, it requires sending the dynamic behaviors to the servers. When the behaviors contain audios and images, the communication overhead will significantly increase than processing them locally in the edge devices.

7 CONCLUSIONS

In this work, we propose CoRec, an efficient deep convolutional neural network-based recommendation framework with edge-cloud collaboration to improve both the accuracy and speed for mobile-web recommendation. Specifically, a novel **convolutional interest network (CIN)** improves the accuracy by modeling the long- and short-term user interest and speeds up the prediction through the parallel-friendly convolutions. To further alleviate the server pressure and increase the serving throughput, we propose a novel device-cloud collaboration strategy of CIN, where the online workloads are reduced by pre-computing and caching the long-term interest representations offline and short-term interest is computed in the device on the fly. In the future, we will investigate the transformer-based CTR prediction models and their edge-cloud collaborative strategies and the training techniques for large-scale and dynamic recommendation systems.

REFERENCES

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. Tensorflow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI16)*. 265–283.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2014. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473* (2014).

[3] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. 2018. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271* (2018).

[4] Deepayan Chakrabarti, Deepak Agarwal, and Vanja Josifovski. 2008. Contextual advertising by combining relevance with click feedback. In *Proceedings of the 17th International Conference on World Wide Web*. 417–426.

[5] Jianxin Chang, Chen Gao, Yu Zheng, Yiqun Hui, Yanan Niu, Yang Song, Depeng Jin, and Yong Li. 2021. Sequential recommendation with graph neural networks. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 378–387.

[6] Olivier Chapelle, Eren Manavoglu, and Romer Rosales. 2014. Simple and scalable response prediction for display advertising. *ACM Trans. Intell. Syst. Technol.* 5, 4 (2014), 1–34.

[7] Cen Chen, Kenli Li, Aijia Ouyang, and Keqin Li. 2018. FlinkCL: An openCL-based in-memory computing architecture on heterogeneous CPU-GPU clusters for big data. *IEEE Trans. Comput.* 67, 12 (2018), 1765–1779.

[8] Cen Chen, Kenli Li, Aijia Ouyang, Zeng Zeng, and Keqin Li. 2018. GFlink: An in-memory computing architecture on heterogeneous CPU-GPU clusters for big data. *IEEE Trans. Parallel Distrib. Syst.* 29, 6 (2018), 1275–1288.

[9] Cen Chen, Kenli Li, Sin G. Teo, Xiaofeng Zou, Keqin Li, and Zeng Zeng. 2020. Citywide traffic flow prediction based on multiple gated spatio-temporal convolutional neural networks. *ACM Trans. Knowl. Discov. Data* 14, 4 (2020), 1–23.

[10] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. 2016. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*. ACM, 7–10.

[11] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555* (2014).

[12] Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. 2017. Language modeling with gated convolutional networks. In *Proceedings of the 34th International Conference on Machine Learning*. JMLR. org, 933–941.

[13] Kushal S. Dave and Vasudeva Varma. 2010. Learning the click-through rate for rare/new ads from similar ads. In *Proceedings of the 33rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 897–898.

[14] Zhou Fang, Dezhi Hong, and Rajesh K. Gupta. 2019. Serving deep neural networks at the cloud edge for vision applications on mobile platforms. In *Proceedings of the 10th ACM Multimedia Systems Conference*. 36–47.

[15] Yufei Feng, Fuyu Lv, Weichen Shen, Menghan Wang, Fei Sun, Yu Zhu, and Keping Yang. 2019. Deep session interest network for click-through rate prediction. *arXiv preprint arXiv:1905.06482* (2019).

[16] Jonas Gehring, Michael Auli, David Grangier, and Yann N. Dauphin. 2016. A convolutional encoder model for neural machine translation. *arXiv preprint arXiv:1611.02344* (2016).

[17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 770–778.

[18] Xinran He, Junfeng Pan, Ou Jin, Tianbing Xu, Bo Liu, Tao Xu, Yanxin Shi, Antoine Atallah, Ralf Herbrich, Stuart Bowers, et al. 2014. Practical lessons from predicting clicks on ads at Facebook. In *Proceedings of the 8th International Workshop on Data Mining for Online Advertising*. 1–9.

[19] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. 2017. Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 4700–4708.

[20] Yakun Huang, Xiuquan Qiao, Pei Ren, Ling Liu, Calton Pu, Schahram Dustdar, and Junliang Chen. 2020. A lightweight collaborative deep neural network for the mobile web in edge cloud. *IEEE Trans. Mob. Comput.* 21, 7 (2020).

[21] Yakun Huang, Xiuquan Qiao, Jian Tang, Pei Ren, Ling Liu, Calton Pu, and Junliang Chen. 2020. DeepAdapter: A collaborative deep learning framework for the mobile web using context-aware network pruning. In *Proceedings of the IEEE Conference on Computer Communications*. IEEE, 834–843.

[22] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. 2016. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099* (2016).

[23] Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor Mudge, Jason Mars, and Lingjia Tang. 2017. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. *ACM SIGARCH Comput. Archit. News* 45, 1 (2017), 615–629.

[24] Guokun Lai, Wei Cheng Chang, Yiming Yang, and Hanxiao Liu. 2017. Modeling long- and short-term temporal patterns with deep neural networks. In *the 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*. 95–104. https://doi.org/10.1145/3209978.3210006

[25] Jianxun Lian, Xiaohuan Zhou, Fuzheng Zhang, Zhongxia Chen, Xing Xie, and Guangzhong Sun. 2018. xDeepFM: Combining explicit and implicit feature interactions for recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1754–1763.

[26] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. 2017. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2117–2125.

[27] Bin Liu, Ruiming Tang, Yingzhi Chen, Jinkai Yu, Huifeng Guo, and Yuzhou Zhang. 2019. Feature generation by convolutional neural network for click-through rate prediction. In *Proceedings of the World Wide Web Conference*. ACM, 1119–1129.

[28] Wantong Lu, Yantao Yu, Yongzhe Chang, Zhen Wang, Chenhui Li, and Bo Yuan. 2021. A dual input-aware factorization machine for CTR prediction. In *Proceedings of the 29th International Conference on International Joint Conferences on Artificial Intelligence*. 3139–3145.

[29] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *J. Mach. Learn. Res.* 9, Nov. (2008), 2579–2605.

[30] Jiachen Mao, Xiang Chen, Kent W. Nixon, Christopher Krieger, and Yiran Chen. 2017. MoDNN: Local distributed mobile computing system for deep neural network. In *Proceedings of the Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1396–1401.

[31] Jiachen Mao, Qing Yang, Ang Li, Hai Li, and Yiran Chen. 2019. MobiEye: An efficient cloud-based video detection system for real-time mobile applications. In *Proceedings of the 56th Annual Design Automation Conference*. 1–6.

[32] H. Brendan McMahan, Gary Holt, David Sculley, Michael Young, Dietmar Ebner, Julian Grady, Lan Nie, Todd Phillips, Eugene Davydov, Daniel Golovin, et al. 2013. Ad click prediction: A view from the trenches. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 1222–1230.

[33] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Proceedings of the Conference on Advances in Neural Information Processing Systems*. 3111–3119.

[34] Richard J. Oentaryo, Ee-Peng Lim, Jia-Wei Low, David Lo, and Michael Finegold. 2014. Predicting response in mobile advertising with hierarchical importance-aware factorization machine. In *Proceedings of the 7th ACM International Conference on Web Search and Data Mining*. 123–132.

[35] Yanru Qu, Han Cai, Kan Ren, Weinan Zhang, Yong Yu, Ying Wen, and Jun Wang. 2016. Product-based neural networks for user response prediction. In *Proceedings of the IEEE 16th International Conference on Data Mining (ICDM)*. IEEE, 1149–1154.

[36] Lara Quijano-Sánchez, Iván Cantador, María E. Cortés-Cediel, and Olga Gil. 2020. Recommender systems for smart cities. *Inf. Syst.* 92 (2020), 101545.

[37] Xukan Ran, Haolianz Chen, Xiaodan Zhu, Zhenming Liu, and Jiasi Chen. 2018. DeepDecision: A mobile deep learning framework for edge video analytics. In *Proceedings of the IEEE Conference on Computer Communications*. IEEE, 1421–1429.

[38] Steffen Rendle. 2010. Factorization machines. In *Proceedings of the IEEE International Conference on Data Mining*. IEEE, 995–1000.

[39] Steffen Rendle. 2012. Factorization machines with libFM. *ACM Trans. Intell. Syst. Technol.* 3, 3 (2012), 1–22.

[40] Matthew Richardson, Ewa Dominowska, and Robert Ragno. 2007. Predicting clicks: Estimating the click-through rate for new ads. In *Proceedings of the 16th International Conference on World Wide Web*. ACM, 521–530.

[41] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 4510–4520.

[42] Surat Teerapittayanon, Bradley McDanel, and Hsiang-Tsung Kung. 2017. Distributed deep neural networks over the cloud, the edge and end devices. In *Proceedings of the IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 328–339.

[43] Ling Yan, Wu-Jun Li, Gui-Rong Xue, and Dingyi Han. 2014. Coupled group lasso for web-scale CTR prediction in display advertising. In *Proceedings of the International Conference on Machine Learning*. 802–810.

[44] Jiaxuan You, Yichen Wang, Aditya Pal, Pong Eksombatchai, Chuck Rosenburg, and Jure Leskovec. 2019. Hierarchical temporal convolutional networks for dynamic recommender systems. In *Proceedings of the World Wide Web Conference*. ACM, 2236–2246.

[45] Feng Yu, Qiang Liu, Shu Wu, Liang Wang, and Tieniu Tan. 2016. A dynamic recurrent model for next basket recommendation. In *Proceedings of the 39th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 729–732.

[46] Fisher Yu, Dequan Wang, Evan Shelhamer, and Trevor Darrell. 2018. Deep layer aggregation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2403–2412.

[47] Minjia Zhang, Samyam Rajbhandari, Wenhan Wang, and Yuxiong He. 2018. DeepCPU: Serving RNN-based deep learning models 10x faster. In *Proceedings of the USENIX Annual Technical Conference (USENIXATC'18)*. 951–965.

[48] Wenjie Zhang, Shunqi Liu, Oktoviano Gandhi, Carlos D. Rodríguez-Gallegos, Hao Quan, and Dipti Srinivasan. 2021. Deep-learning-based probabilistic estimation of solar PV soiling loss. *IEEE Trans. Sustain. Energy* 12, 4 (2021), 2436–2444.

[49] Wenjie Zhang, Hao Quan, and Dipti Srinivasan. 2018. An improved quantile regression neural network for probabilistic load forecasting. *IEEE Trans. Smart Grid* 10, 4 (2018).

[50] Wenjie Zhang, Hao Quan, and Dipti Srinivasan. 2018. Parallel and reliable probabilistic load forecasting via quantile regression forest and quantile determination. *Energy* 160 (2018), 810–819.

[51] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep interest evolution network for click-through rate prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 5941–5948.

[52] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 1059–1068.

Received 7 August 2021; revised 21 January 2022; accepted 11 March 2022